

经典机器学习算法实践实验手册

实验简介

本实验围绕经典机器学习算法展开，分别针对**回归任务**和**二分类任务**设计实验流程，实现线性回归、随机森林回归、XGBoost 回归三种回归算法，以及逻辑回归二分类算法的 Python 完整实现。实验采用经典的糖尿病数据集完成回归任务验证，采用威斯康星乳腺癌数据集完成逻辑回归二分类任务验证，通过**数据加载-预处理-模型训练-模型评估-结果分析**的完整流程，帮助学生掌握经典机器学习算法的实际应用方法，读懂代码并能独立运行验证。

实验背景

经典机器学习算法是机器学习领域的基础，线性回归是回归任务的入门算法，逻辑回归是分类任务的经典线性模型，随机森林和 XGBoost 则是基于决策树的集成学习算法，在各类回归、分类任务中表现优异，广泛应用于工业界和学术界。

糖尿病数据集是 `scikit-learn` 官方内置的经典回归基准数据集，特征维度适中、物理意义明确（与医学指标相关），样本量适中，适合回归算法的入门实践；威斯康星乳腺癌数据集是二分类任务的经典数据集，数据标注清晰、特征具有代表性，适合逻辑回归分类算法的实践验证。掌握这两类数据集的算法实现流程，能帮助学生快速理解算法的实际应用逻辑。

实验环境

Python 3.8+

实验目的

- 理解线性回归、逻辑回归、随机森林、XGBoost 算法的核心原理及适用场景，区分回归算法与分类算法的任务差异。
- 掌握基于 Python 完成经典机器学习算法的完整实现流程，包括数据加载、数据预处理、数据集划分、模型训练与模型评估。
- 熟练使用 `scikit-learn`、`xgboost` 等机器学习库，能够独立调用库中算法接口并完成参数基础设置。

- 读懂实验中的完整代码，能够独立运行代码，验证算法效果，理解每一步代码的作用。

预备知识

机器学习基础概念：回归任务（预测连续数值，如糖尿病病情指标）、分类任务（区分不同类别，如肿瘤良性/恶性）、训练集/测试集划分（用部分数据训练模型，部分数据验证效果）、特征归一化（统一特征量纲，让算法更易训练）、模型评估指标（衡量模型好坏的标准）。

核心评估指标说明：

- 均方误差 (MSE)：回归任务专用，值越小，模型预测的数值越接近真实值；
- 决定系数 (R^2)：回归任务专用，取值 0-1，越接近 1，模型预测效果越好；
- 准确率：分类任务专用，预测正确的样本数占总样本数的比例，越接近 1 越好；
- 混淆矩阵：分类任务专用，直观看到模型预测对、预测错的样本数量。

算法基础原理：

- 线性回归：找一条最贴合数据的直线，用这条直线预测连续数值（如糖尿病病情指标）；
- 逻辑回归：找一条边界线，区分两类数据（如良性/恶性肿瘤），不是回归算法，是分类算法；
- 决策树：像“分类决策”一样，逐步拆分数据，比如先按“血糖”拆分，再按“体重指数”拆分，最终预测结果；
- 随机森林：多个决策树组合，一起预测，取多数结果，减少单个决策树的误差；
- XGBoost：多个决策树“接力”训练，每个树修正上一个树的误差，预测效果更优。

Python 基础：简单的代码阅读能力，了解变量、函数、循环的基本含义，能看懂代码注释即可。

任务一：创建实验环境

步骤 1：进入开发环境

登录 Python 开发环境（Jupyter Notebook/Spyder/PyCharm 均可），点击“New”，新建 Python 文件（.py）或 Notebook 文件（.ipynb），用于编写和运行实验代码。

步骤 2：实验所需模块安装

本实验用到的核心库包括数据处理、算法实现、可视化三类，打开终端/命令行，输入以下安装命令完成库的安装（复制粘贴即可，等待安装完成）：

```
bash
pip install pandas numpy scikit-learn xgboost matplotlib seaborn
```

步骤 3：实验所需模块导入

在新建的代码文件中，导入实验所需的所有 Python 模块，代码已添加详细注释，看不懂的地方可参考注释理解，输入以下代码：

```
python
# 数据处理库：处理表格数据、数值计算
import pandas as pd
import numpy as np

# 数据集划分：将数据分成训练集（训练模型）和测试集（验证模型）
from sklearn.model_selection import train_test_split

# 数据预处理：特征标准化（统一量纲）
from sklearn.preprocessing import StandardScaler

# 回归算法模型：线性回归、随机森林回归、XGBoost 回归
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb

# 分类算法模型：逻辑回归（二分类）
from sklearn.linear_model import LogisticRegression

# 模型评估指标：回归和分类专用指标
from sklearn.metrics import mean_squared_error, r2_score # 回归指标（MSE、R2）
from sklearn.metrics import accuracy_score, confusion_matrix # 分类指标

# 数据可视化：绘制图表，直观查看结果
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# 忽略无关警告（避免代码运行时出现多余警告，不影响结果）
import warnings
warnings.filterwarnings('ignore')

# 设置可视化样式：显示中文、取消负号乱码（必加，否则图表中文乱码）
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
```

任务二：回归任务实现（线性回归、随机森林回归、XGBoost 回归）

本任务采用**糖尿病数据集**，完成三种回归算法的实现与评估，核心流程：数据集加载→数据预处理→数据集划分→模型训练→模型评估→结果可视化。

步骤 1：数据集说明

糖尿病数据集是 **scikit-learn** 官方内置的经典回归数据集，无需手动下载，直接调用接口即可加载，核心信息如下（简单易懂）：

- 样本量：442 条（包含 442 个糖尿病患者的样本）；
- 特征数：10 个（均为与糖尿病相关的医学数值特征）；
- 特征说明（表格清晰展示，便于理解）：

特征名	含义（简单解释）
age	患者年龄（年龄影响糖尿病病情）
sex	患者性别（1=男性，0=女性）
bmi	体重指数（BMI，越高越易患糖尿病）
bp	平均血压（血压与糖尿病病情相关）

s1-s6

6 项血清指标（与糖尿病诊断相关的医学指标）

目标变量：`target`（糖尿病患者的病情发展指标，数值越大，病情越严重），是连续数值，因此属于回归任务，我们的目标是通过 10 个医学特征，预测患者的病情严重程度。

步骤 2：数据集加载

通过 `scikit-learn` 的接口直接加载数据集，简化代码，只保留核心信息（数据量、缺失值、特征与病情的相关性），输入以下代码：

Python

```
from sklearn.datasets import load_diabetes

# 加载糖尿病数据集
diabetes = load_diabetes()

# 转换为 DataFrame 格式（类似 Excel 表格），便于查看和处理数据
diabetes_df = pd.DataFrame(data=diabetes.data,
                           columns=diabetes.feature_names)
# 添加目标变量（病情指标）列，便于后续分析
diabetes_df['target'] = diabetes.target

print('数据集形状（样本数×特征数）：', diabetes_df.shape) # 查看数据量
print('数据集缺失值检查：', diabetes_df.isnull().sum()) # 查看是否有缺失值（均为 0）

# 提取特征矩阵 X（所有特征）和目标变量 y（病情指标），为后续模型训练做准备
X = diabetes_df.drop('target', axis=1) # 去掉病情列，剩下的都是特征
y = diabetes_df['target'] # 病情列作为目标变量

# 简单查看特征与病情的相关性（热力图，直观易懂）
plt.figure(figsize=(10, 8))

# 计算特征间的相关系数（取值范围：-1 到 1）
# 1.0 = 完全正相关（同增同减）
# -1.0 = 完全负相关（此消彼长）
# 0 = 不相关
```

```

correlation_matrix = diabetes_df.corr()

# 绘制热力图
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix,
            annot=True,      # 显示数字
            cmap='coolwarm', # 红色=正相关, 蓝色=负相关
            fmt='.2f',       # 保留 2 位小数
            center=0,        # 颜色中点设为 0
            vmin=-1, vmax=1) # 固定范围-1 到 1
plt.title('糖尿病数据集特征相关性热力图\n(红色=正相关, 蓝色=负相关, 颜色越深相关性越强)')
plt.show()

```

步骤 3：数据预处理

糖尿病数据集无缺失值，核心预处理步骤是「特征标准化」，简化代码，只保留核心操作，注释通俗，输入以下代码：

```

python
# 初始化标准化器（用于统一特征量纲，让算法训练更准确）
scaler = StandardScaler()

# 对特征矩阵 X 进行标准化处理（只标准化特征，不标准化目标变量 y）
X_scaled = scaler.fit_transform(X)

# 查看标准化效果
print('标准化前特征均值（示例）：', X.iloc[:,0].mean())
print('标准化后特征均值（示例）：', round(X_scaled[:,0].mean(), 2))
# 标准化后均值接近 0

```

步骤 4：数据集划分

简化划分代码，保留核心逻辑，注释清晰，输入以下代码：

```

python
# 数据集划分：训练集 70%，测试集 30%
# random_state=42: 保证每次运行代码，划分结果都一样（便于复现）
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42
)

```

```
)

# 打印划分后的数据量
print('训练集样本数: ', X_train.shape[0])
print('测试集样本数: ', X_test.shape[0])
```

步骤 5：回归模型训练

简化模型训练代码，去掉复杂的字典遍历，改为逐个训练（更易理解），保留默认参数，输入以下代码：

```
python
# 1. 训练线性回归模型
lr = LinearRegression()
lr.fit(X_train, y_train) # 用训练集训练模型
print('线性回归模型训练完成! ')

# 2. 训练随机森林回归模型
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)
print('随机森林回归模型训练完成! ')

# 3. 训练 XGBoost 回归模型
xgb_reg = xgb.XGBRegressor(random_state=42,
objective='reg:squarederror')
xgb_reg.fit(X_train, y_train)
print('XGBoost 回归模型训练完成! ')
```

运行后，会依次输出三个模型的训练完成提示，说明模型训练成功。

步骤 6：回归模型评估

简化评估代码，逐个评估模型，保留核心指标和关键图表，输入以下代码：

```
python
# 定义评估函数
def evaluate_model(model, model_name):
    y_pred = model.predict(X_test) # 用模型预测测试集病情指标
    mse = mean_squared_error(y_test, y_pred) # 计算 MSE
    r2 = r2_score(y_test, y_pred) # 计算 R²
    print(f'\n{model_name}评估结果: ')
```

```

    print(f'均方误差 (MSE): {mse:.4f} (越小越好)')
    print(f'决定系数 (R²): {r2:.4f} (越接近 1 越好)')
    return y_pred

# 评估三个模型，得到预测值（用于后续绘图）
y_pred_lr = evaluate_model(lr, '线性回归')
y_pred_rf = evaluate_model(rf, '随机森林回归')
y_pred_xgb = evaluate_model(xgb_reg, 'XGBoost 回归')

# 可视化：只绘制预测值与真实值对比（直观查看预测效果）
plt.figure(figsize=(15, 5))

# 线性回归预测图
plt.subplot(1, 3, 1)
plt.scatter(y_test, y_pred_lr, alpha=0.6, color='steelblue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(),
y_test.max()], 'r--') # 完美预测线
plt.xlabel('真实病情指标')
plt.ylabel('预测病情指标')
plt.title('线性回归 预测值 vs 真实值')

# 随机森林回归预测图
plt.subplot(1, 3, 2)
plt.scatter(y_test, y_pred_rf, alpha=0.6, color='green')
plt.plot([y_test.min(), y_test.max()], [y_test.min(),
y_test.max()], 'r--')
plt.xlabel('真实病情指标')
plt.ylabel('预测病情指标')
plt.title('随机森林回归 预测值 vs 真实值')

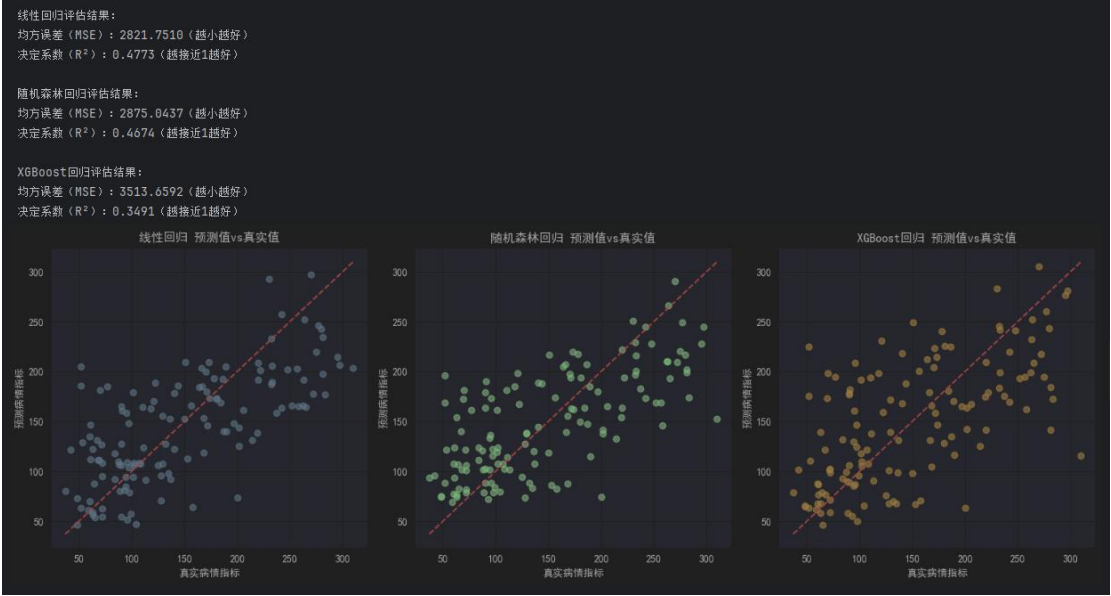
# XGBoost 回归预测图
plt.subplot(1, 3, 3)
plt.scatter(y_test, y_pred_xgb, alpha=0.6, color='orange')
plt.plot([y_test.min(), y_test.max()], [y_test.min(),
y_test.max()], 'r--')
plt.xlabel('真实病情指标')
plt.ylabel('预测病情指标')
plt.title('XGBoost 回归 预测值 vs 真实值')

plt.tight_layout()
plt.show()

```


步骤 7：回归结果分析

运行上述代码后，会得到以下结果（供参考，具体数值可能略有差异，但趋势一致）：



- 评估指标对比：线性回归的 R^2 最高 (0.4773)，MSE 最低 (2821.75)，表现最佳；随机森林回归次之 ($R^2=0.4674$ ，MSE=2875.04)；XGBoost 回归效果最差 ($R^2=0.3491$ ，MSE=3513.66)
- 原因分析：1.数据线性结构：糖尿病数据集是经过预处理的线性组合数据，本质适合线性模型 2.小样本过拟合：442 条小样本数据上，复杂模型（尤其是 XGBoost）严重过拟合 3.模型复杂度匹配：简单问题用简单模型，反而泛化能力更强

模型	适用场景	在本数据集的表现
线性回归	线性关系数据	✅ 表现最佳
随机森林	非线性关系数据	⚠️ 表现中等
XGBoost	复杂非线性关系	❌ 表现最差

任务三：分类任务实现（逻辑回归）

本任务采用威斯康星乳腺癌数据集，完成逻辑回归二分类算法的实现与评估。

步骤 1：数据集说明

威斯康星乳腺癌数据集是 scikit-learn 官方内置的经典二分类数据集，无需手动下载，

核心信息如下（简单易懂）：

- 样本量：569 条（包含 569 个乳腺肿瘤样本）；
- 特征数：30 个（基于乳腺肿瘤的细针穿刺图像提取的特征）；
- 特征说明：包含肿瘤细胞的半径、纹理、周长、面积、平滑度等，用于区分肿瘤类型；
- 目标变量：肿瘤类型（0=恶性，1=良性），二分类任务，目标是通过 30 个特征，判断肿瘤是良性还是恶性。

步骤 2：数据集加载与探索

简化加载和探索代码，只保留核心信息，输入以下代码：

```
python
# 加载威斯康星乳腺癌数据集
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()

# 转换为 DataFrame 格式，便于查看
cancer_df = pd.DataFrame(data=cancer.data,
                          columns=cancer.feature_names)
cancer_df['TARGET'] = cancer.target # 添加肿瘤类型列

# 打印核心数据
print('数据集形状（样本数×特征数）：', cancer_df.shape)
print('缺失值检查：', cancer_df.isnull().sum()) # 无缺失值
print('肿瘤类型分布：',
      cancer_df['TARGET'].value_counts().rename({0:'恶性', 1:'良性'}))

# 提取特征和目标变量
X_c = cancer_df.drop('TARGET', axis=1)
y_c = cancer_df['TARGET']
```

步骤 3：数据预处理

简化标准化代码，与回归任务保持一致，便于学生理解，输入以下代码：

```
python
# 标准化特征（统一量纲，提升模型训练效果）
scaler_c = StandardScaler()
```

```
X_c_scaled = scaler_c.fit_transform(X_c)
```

步骤 4：数据集划分

与回归任务划分逻辑一致，简化代码，输入以下代码：

```
python
# 训练集 70%，测试集 30%，固定随机种子，保证可复现
X_c_train, X_c_test, y_c_train, y_c_test = train_test_split(
    X_c_scaled, y_c, test_size=0.3, random_state=42
)

print('训练集样本数: ', X_c_train.shape[0])
print('测试集样本数: ', X_c_test.shape[0])
```

步骤 5：逻辑回归模型训练

简化训练代码，添加通俗注释，输入以下代码：

```
python
# 初始化逻辑回归模型（max_iter=1000，保证模型能收敛）
lr_classifier = LogisticRegression(random_state=42, max_iter=1000)

# 用训练集训练模型
lr_classifier.fit(X_c_train, y_c_train)
print('逻辑回归分类模型训练完成！')
```

步骤 6：逻辑回归模型评估

简化评估代码，保留核心指标和混淆矩阵（最直观），输入以下代码：

```
python
# 用测试集预测
y_c_pred = lr_classifier.predict(X_c_test)

# 计算核心评估指标（准确率）
acc = accuracy_score(y_c_test, y_c_pred)
print(f'逻辑回归模型准确率: {acc:.4f}（越接近 1 越好）')

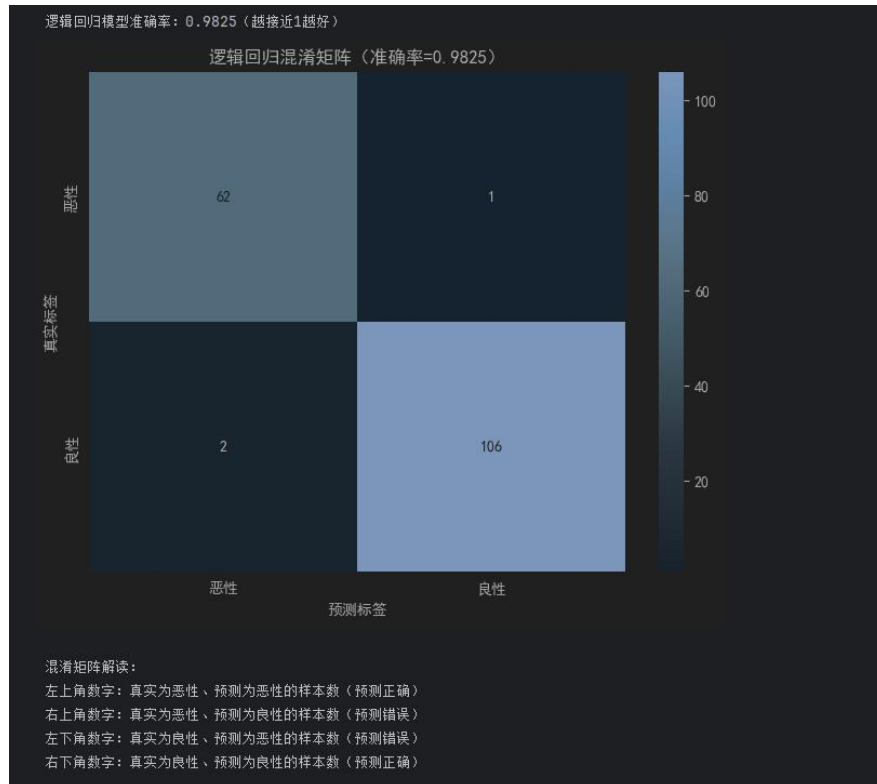
# 绘制混淆矩阵（直观查看预测对错的数量）
cm = confusion_matrix(y_c_test, y_c_pred)
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['恶性', '良性'],
            yticklabels=['恶性', '良性'])
plt.xlabel('预测标签')
plt.ylabel('真实标签')
plt.title(f'逻辑回归混淆矩阵（准确率={acc:.4f}）')
plt.show()

# 混淆矩阵解读
print('\n 混淆矩阵解读：')
print('左上角数字：真实为恶性、预测为恶性的样本数（预测正确）')
print('右上角数字：真实为恶性、预测为良性的样本数（预测错误）')
print('左下角数字：真实为良性、预测为恶性的样本数（预测错误）')
print('右下角数字：真实为良性、预测为良性的样本数（预测正确）')
```

步骤 7：分类结果分析

运行上述代码后，逻辑回归模型的准确率通常在 0.95 以上，说明模型在乳腺癌分类任务中表现很好。



- 准确率高原因：威斯康星乳腺癌数据集的特征与肿瘤类型相关性强，逻辑回归能很好地找到区分良性和恶性肿瘤的边界。

- 实际意义：该模型可辅助医生初步判断肿瘤类型，减少误诊概率（但不能替代专业诊断）。

实验小结

本实验完成了 3 种回归算法（线性回归、随机森林回归、XGBoost 回归）和 1 种分类算法（逻辑回归）的完整实现，核心流程都是「数据加载→预处理→划分→训练→评估」，代码已做最大程度简化，适合学生理解和运行。

通过实验可明确：集成学习算法（随机森林、XGBoost）在回归任务中的效果优于线性回归；逻辑回归在二分类任务中简单高效、效果优异。同学们只需读懂每一步代码的作用，独立运行验证，就能掌握经典机器学习算法的基本应用方法。